

SRI VASAVI ENGINEERING COLLEGE

(AUTONOMOUS)
PEDATADEPALLI, TADEPALLIGUDEM.



Certificate

This is to certify that this is a bonafide record of practical work done in **Full Stack Development-II** by Mr./Miss. _____ bearing Roll No. _____ **Computer Science and Engineering** Branch of **V Semester** during the academic year 2026 to 2027.

No. of Experiments Done: 26

Signature of the Faculty

Head of the Department

INDEX

S.No	Name of the Experiments	Date		Page	Remarks
		Performed	Submitted		
1.	Develop a program to define a route, Handling Routes, Route Parameters, Query Parameters and URL building.			1	
2	Develop a program to accept data, retrieve data and delete a specified resource using HTTP methods.			3	
3	Develop a program to show the working of middleware.			7	
4	Develop a program using templating engine.			8	
5	Develop a program to work with form data.			9	
6	Develop a program for session management using cookies and sessions.			11	
7	Develop a program for user authentication.			13	
8	Develop a program to connect MongoDB database using Mongoose and perform CRUD operations.			16	
9	Develop a program for single page application using RESTful APIs.			18	
10	Develop a program to render HTML to a web page.			20	
11	Develop a program for writing markup with JSX.			22	
12	Develop a program for creating and nesting components (function and class).			24	

13	Develop a program to work with props and states			26	
14	Develop a program to add styles (CSS & Sass Styling) and display data.			28	
15	Develop a program for responding to events.			32	
16	Develop a program for conditional rendering.			34	
17	Develop a program for rendering lists.			36	
18	Develop a program for working with different form fields using React forms.			37	
19	Develop a program for routing to different pages using React Router.			41	
20	Develop a program for updating the screen.			44	
21	Develop a program to understand the importance of using Hooks.			46	
22	Develop a program for sharing data between components.			50	
23	Install MongoDB and configure ATLAS.			52	
24	Develop MongoDB queries to perform CRUD operations on document using insert(), find(), update(), remove().			53	
25	Develop MongoDB queries to Create and drop databases and collections.			56	
26	Develop MongoDB queries to work with records using find(), limit(), sort(), createIndex(), aggregate().			57	

Experiment-1

1. Develop a program to define a route, Handling Routes, Route Parameters, Query Parameters and URL building.

```
const express = require("express");
const app = express();
const PORT = 5000;
app.get("/", (req, res) => {
    res.send("Welcome to ExpressJS Routing");
});
app.get("/about", (req, res) => {
    res.send("This is the About Page");
});
app.get("/contact", (req, res) => {
    res.send("This is the Contact Page");
});
app.get("/student/details", (req, res) => {
    res.send("Student Details Page");
});
app.get("/student/:id", (req, res) => {
    const studentId = req.params.id;
    res.send(`Student ID: ${studentId}`);
});
app.get("/student/:id/:name", (req, res) => {
    const studentId = req.params.id;
    const studentName = req.params.name;
    res.send(`Student ID: ${studentId}, Student Name: ${studentName}`);
});
app.get("/search", (req, res) => {
    const name = req.query.name;
    res.send(`Searching for student: ${name}`);
});
```

```
app.get("/students", (req, res) => {
  const branch = req.query.branch;
  const year = req.query.year;
  res.send(`Branch: ${branch}, Year: ${year}`);
});

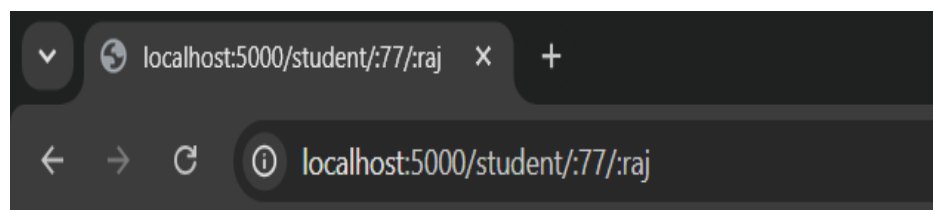
app.get("/build-student-url", (req, res) => {
  const id = 101;
  const name = "Ravi";
  const studentURL = `/student/${id}/${name}`;
  res.send(`Generated Student URL: ${studentURL}`);
});

app.get("/build-search-url", (req, res) => {
  const branch = "CSE";
  const year = 3;
  const searchURL = `/students?branch=${branch}&year=${year}`;
  res.send(`Generated Search URL: ${searchURL}`);
});

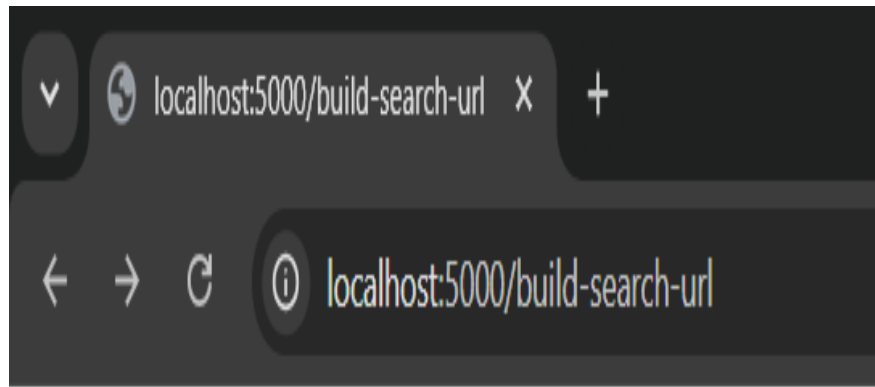
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

OUTPUT:

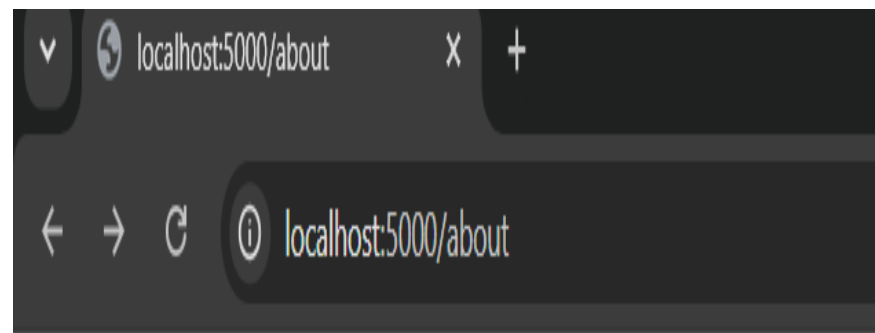
Server is running at <http://localhost:5000>



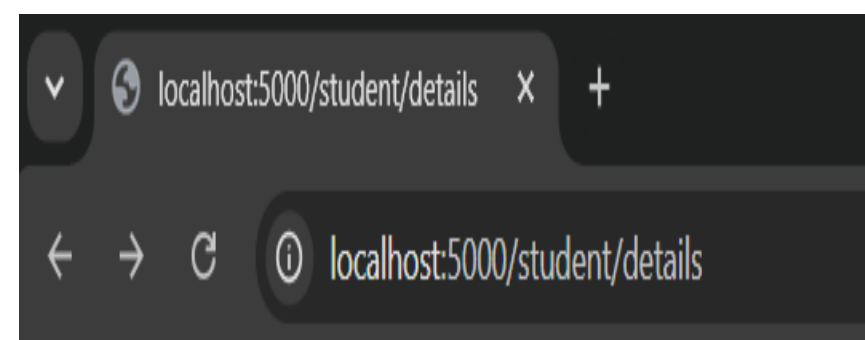
Student ID: :77, Student Name: :raj



Generated Search URL: /students?branch=CSE&year=3



This is the About Page



Student Details Page

2. Develop a program to accept data, retrieve data and delete a specified resource using http methods.

```
const express = require("express");
const app = express();
const PORT = 5000;
app.use(express.json());
let students = [
  {
    id: 101,
    name: "Ravi",
    branch: "CSE"
  },
  {
    id: 102,
    name: "Sita",
    branch: "ECE"
  },
  {
    id: 103,
    name: "Rahul",
    branch: "IT"
  }
];
app.post("/students", (req, res) => {
  const student = req.body;
  students.push(student);
  res.status(201).json({
    message: "Student added successfully",
    student: student
  });
});
app.get("/students", (req, res) => {
```

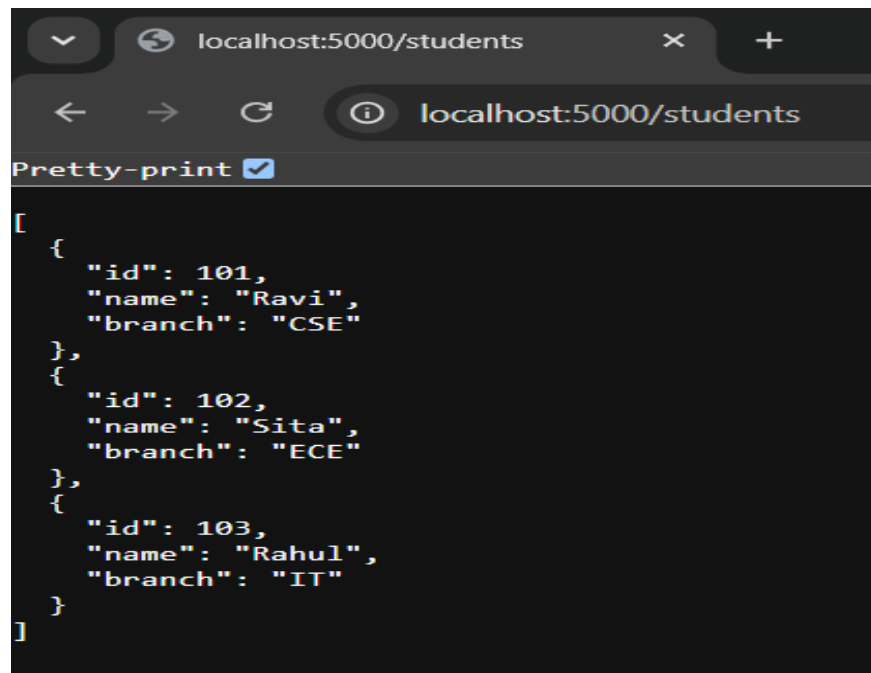
```

    res.json(students);
  });
  app.get("/students/:id", (req, res) => {
    const id = Number(req.params.id);
    const student = students.find(student => student.id === id);
    if (!student) {
      return res.status(404).json({
        message: "Student not found"
      });
    }
    res.json(student);
  });
  app.delete("/students/:id", (req, res) => {
    const id = Number(req.params.id);
    const index = students.findIndex(student => student.id === id);
    if (index === -1) {
      return res.status(404).json({
        message: "Student not found"
      });
    }
    students.splice(index, 1);
    res.json({
      message: "Student deleted successfully"
    });
  });
  app.listen(PORT, () => {
    console.log(`Server running on http://localhost:${PORT}`);
  });

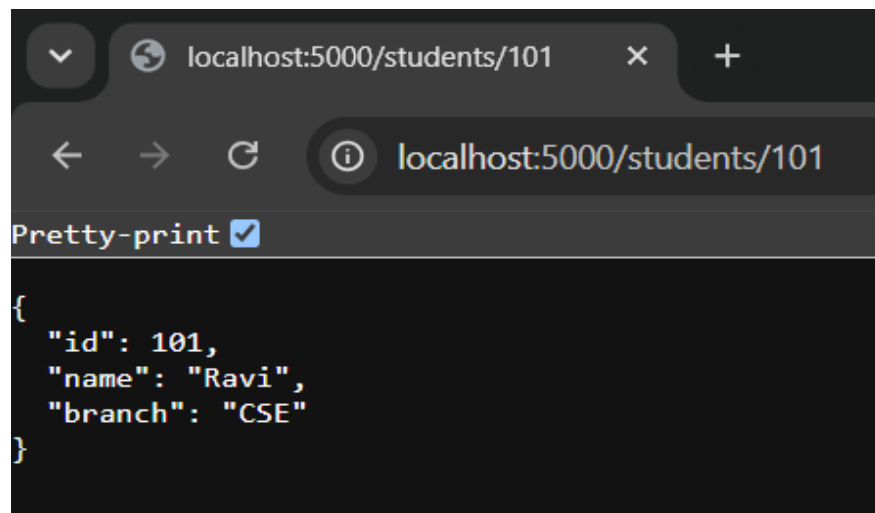
```

OUTPUT:

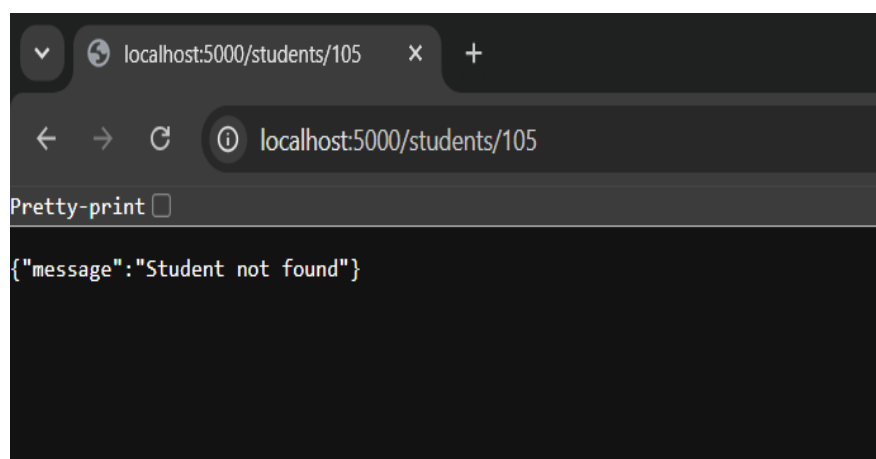
Server running on <http://localhost:5000>



```
[
  {
    "id": 101,
    "name": "Ravi",
    "branch": "CSE"
  },
  {
    "id": 102,
    "name": "Sita",
    "branch": "ECE"
  },
  {
    "id": 103,
    "name": "Rahul",
    "branch": "IT"
  }
]
```



```
{
  "id": 101,
  "name": "Ravi",
  "branch": "CSE"
}
```



```
{"message": "Student not found"}
```

3. Develop a program to show the working of middleware.

```
const express = require("express");
const app = express();
const PORT = 5000;

app.use((req, res, next) => {
  console.log("Middleware 1 executed");
  next();
});

app.get("/", (req, res) => {
  console.log("Route 1 executed");
  res.send("Home Page");
});

app.use((req, res, next) => {
  console.log("Middleware 2 executed");
  next();
});

app.get("/about", (req, res) => {
  console.log("Route 2 executed");
  res.send("About Page");
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

OUTPUT:

Server running at <http://localhost:5000>

Middleware 1 executed

Route 1 executed

Middleware 1 executed

Middleware 2 executed

Home page

Middleware 1 executed

Middleware 2 executed

Route 2 executed

About page

Experiment - 2

1. Develop a program using templating engine.

1. server.js

```
const express = require("express");
const app = express();
const PORT = 5000;
app.set("view engine", "ejs");
app.get("/", (req, res) => {
  const name = "Ravi";
  const branch = "CSE";
  res.render("home", { name, branch });
});
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

2. views/home.ejs

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Details</title>
</head>
<body>
  <h1>Student Details</h1>
  <h2>Name: <%= name %></h2>
  <h2>Branch: <%= branch %></h2>
</body>
</html>
```

OUTPUT:

Student Details

Name: Ravi

Branch: CSE

2. Develop a program to work with form data.

Server.js

```
const express = require("express");
const app = express();
const PORT = 5000;
app.use(express.urlencoded({ extended: true }));

app.get("/", (req, res) => {
  res.sendFile(__dirname + "/form.html");
});

app.post("/submit", (req, res) => {
  const name = req.body.name;
  const email = req.body.email;
  const branch = req.body.branch;
  res.send(`Name: ${name}<br>Email: ${email}<br>Branch: ${branch}`);
});

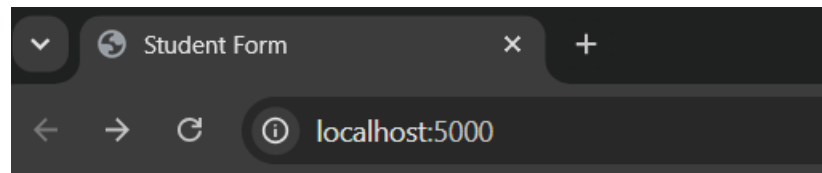
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Form.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Student Form</title>
  </head>
  <body>
    <h1>Student Registration Form</h1>
    <form action="/submit" method="POST">
      Name:
      <input type="text" name="name">
```

```
<br><br>
Email:
<input type="email" name="email">
<br><br>
Branch:
<input type="text" name="branch">
<br><br>
<button type="submit">Submit</button>
</form>
</body>
</html>
```

OUTPUT:



Student Registration Form

Name:

Email:

Branch:

Name: raj

Email: raj@gmail.com

Branch: cst

Experiment-3

1. Develop a program for session management using cookies and sessions.

```
const express = require("express");
const session = require("express-session");
const app = express();

app.use(session({
  secret: "secret",
  resave: false,
  saveUninitialized: true
}));

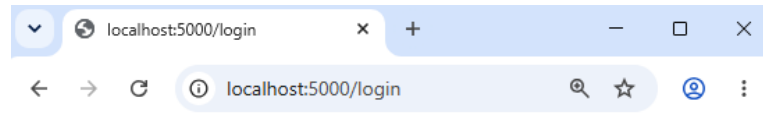
app.get("/login", (req, res) => {
  req.session.name = "Ravi";
  res.cookie("user", "Ravi");
  res.send("Login Successful");
});

app.get("/profile", (req, res) => {
  res.send("Welcome " + req.session.name);
});

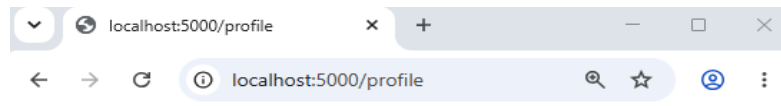
app.get("/logout", (req, res) => {
  req.session.destroy();
  res.clearCookie("user");
  res.send("Logout Successful");
});

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

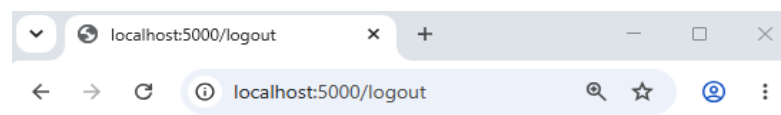
OUTPUT:



Login
Successful



Welcome Ravi

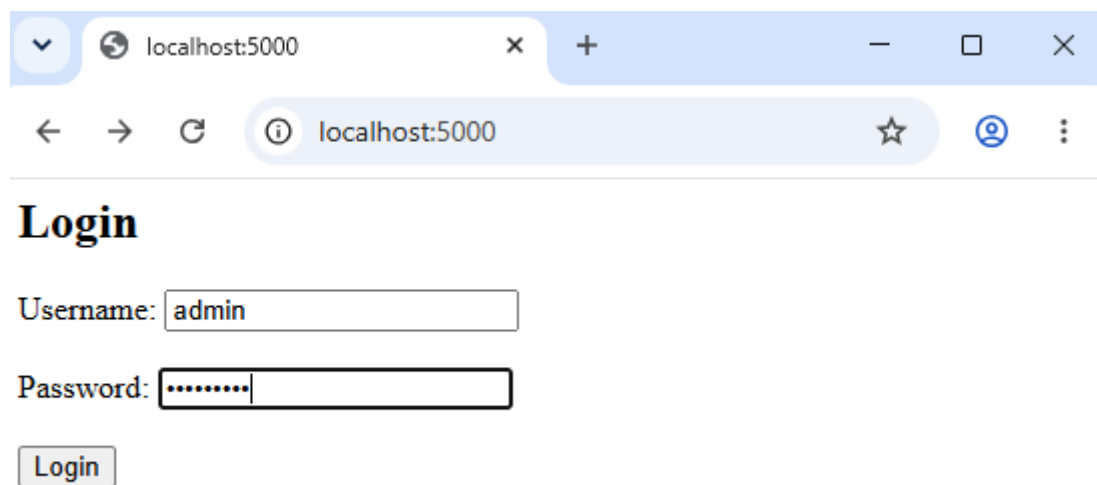


Logout
Successful

2. Develop a program for user authentication

```
const express = require("express");
const app = express();
app.use(express.urlencoded({ extended: true }));
const username = "admin";
const password = "1234";
app.get("/", (req, res) => {
  res.send(`
    <h2>Login</h2>
    <form method="POST" action="/login">
      Username: <input type="text" name="username"><br><br>
      Password: <input type="password" name="password"><br><br>
      <button type="submit">Login</button>
    </form>
  `);
});
app.post("/login", (req, res) => {
  if (req.body.username === username && req.body.password === password) {
    res.send("Login Successful");
  } else {
    res.send("Invalid Username or Password");
  }
});
app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

OUTPUT:



A screenshot of a web browser window. The address bar shows 'localhost:5000'. The page title is 'Login'. Below the title, there are two input fields: 'Username:' with the value 'admin' and 'Password:' with masked characters '.....'. A 'Login' button is located below the password field.

Login

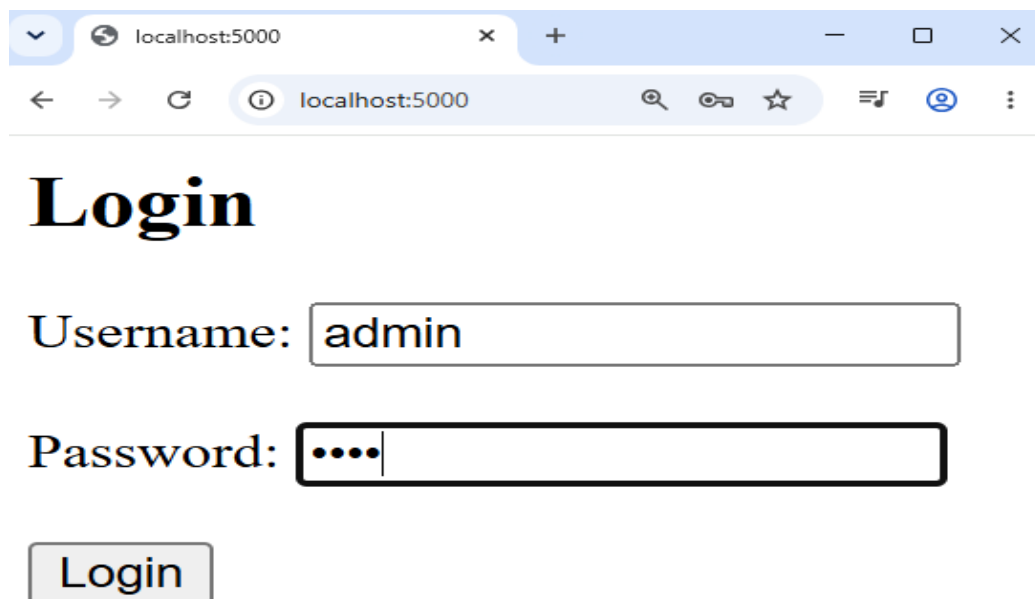
Username:

Password:



A screenshot of a web browser window. The address bar shows 'localhost:5000/login'. The page displays the text 'Invalid Username or Password' in a large, serif font.

Invalid Username or Password



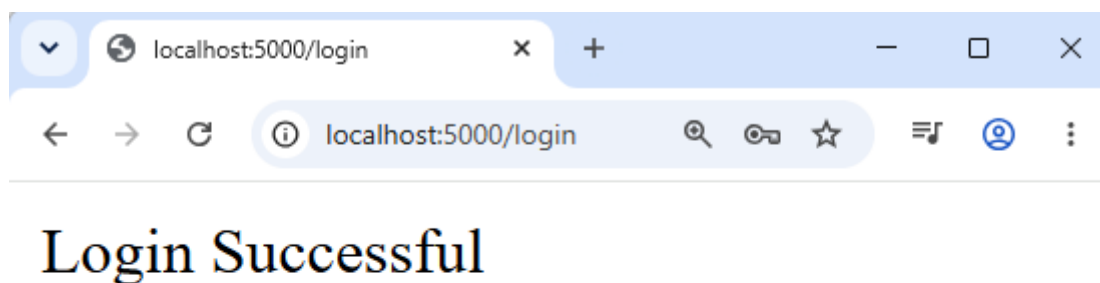
A screenshot of a web browser window. The address bar shows 'localhost:5000'. The page has a large 'Login' heading. Below it, there is a 'Username:' label followed by a text input field containing 'admin'. Below that is a 'Password:' label followed by a password input field with four dots. At the bottom is a 'Login' button.

Login

Username:

Password:

Login



A screenshot of a web browser window. The address bar shows 'localhost:5000/login'. The page displays the text 'Login Successful'.

Login Successful

Experiment-4

1. Develop a program to connect MongoDB database using Mongoose and perform CRUD operations.

```
const express = require("express");
const mongoose = require("mongoose");
const app = express();
app.use(express.json());
mongoose.connect("mongodb://127.0.0.1:27017/studentdb");
const Student = mongoose.model("Student", {
  name: String,
  branch: String
});
app.post("/add", async (req, res) => {
  await Student.create(req.body);
  res.send("Student Added");
});
app.get("/students", async (req, res) => {
  const data = await Student.find();
  res.json(data);
});
app.put("/update/:id", async (req, res) => {
  await Student.findByIdAndUpdate(req.params.id, req.body);
  res.send("Student Updated");
});
app.delete("/delete/:id", async (req, res) => {
  await Student.findByIdAndDelete(req.params.id);
  res.send("Student Deleted");
});
app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

OUTPUT:

Server running on port 5000

Student Added

[{"name":"Alex","branch":"Computer Science"}]

Student Updated

Student Deleted

2. Develop a program for single page application using RESTful APIs.

Server.js

```
const express = require("express");
const app = express();
app.use(express.json());

let students = [
  { id: 1, name: "Ravi", branch: "CSE" },
  { id: 2, name: "Sita", branch: "ECE" }
];

app.get("/api/students", (req, res) => {
  res.json(students);
});

app.post("/api/students", (req, res) => {
  students.push(req.body);
  res.json({ message: "Student Added" });
});

app.delete("/api/students/:id", (req, res) => {
  students = students.filter(s => s.id !== req.params.id);
  res.json({ message: "Student Deleted" });
});

app.get("/", (req, res) => {
  res.send(`
    <h2>Student SPA</h2>
    <button onclick="loadStudents()">View Students</button>
    <div id="data"></div>

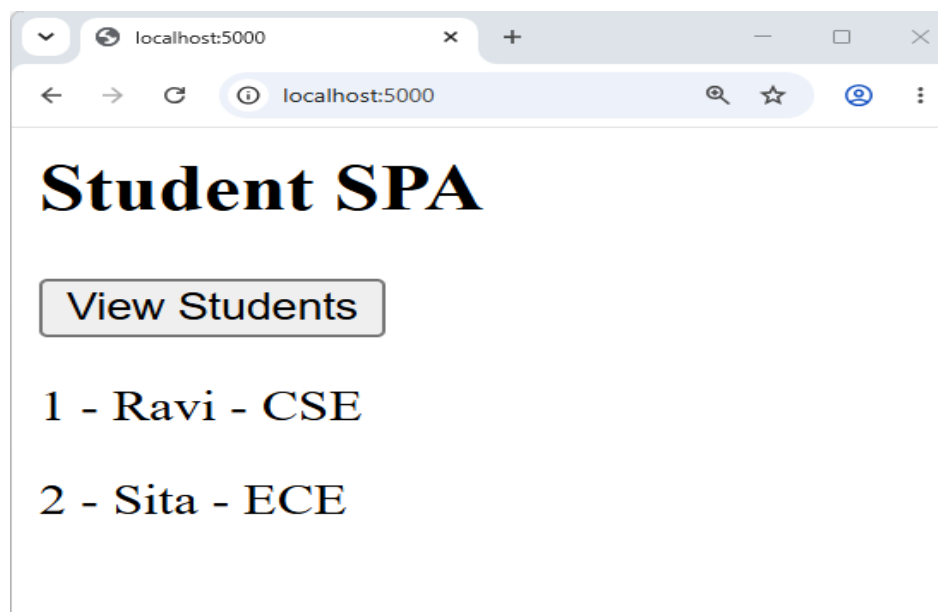
    <script>
      async function loadStudents() {
```

```
const response = await fetch("/api/students");
const students = await response.json();

document.getElementById("data").innerHTML =
  students.map(s =>
    "<p>" + s.id + " - " + s.name + " - " + s.branch + "</p>"
  ).join("");
}
</script>
`);
});

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

OUTPUT:



Experiment – 5

1. Develop a program to render HTML to a web page.

Index.html

```
<!DOCTYPE html>

<html>

<head>

  <title>React App</title>

</head>

<body>

  <div id="root"></div>

</body>

</html>
```

App.js

```
import React from 'react';

function App() {

  return (

    <div>

      <h1>Welcome to ReactJS</h1>

      <h2>HTML Rendering</h2>

      <p>This is a ReactJS web page.</p>

      <p>React renders these HTML elements in the browser.</p>

    </div>

  );

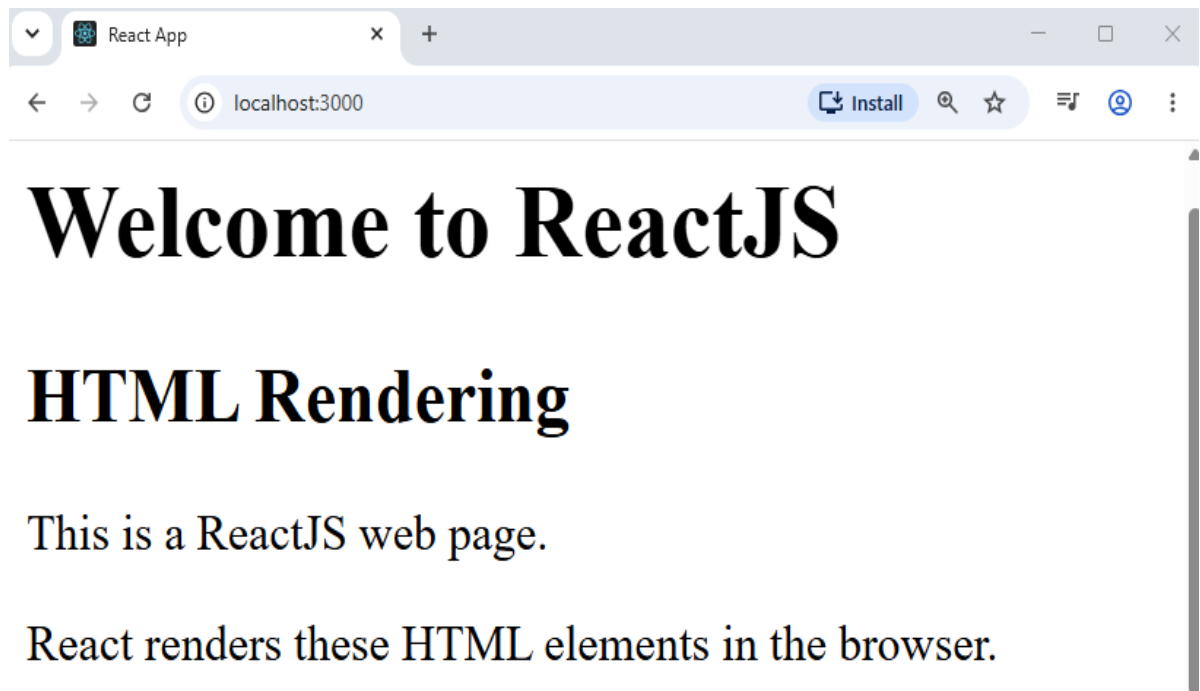
}

export default App;
```

index.js

```
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import App from './App';  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(  <App />);
```

OUTPUT:



2. Develop a program for writing markup with JSX.

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>React App</title>
</head>
<body>
  <div id="root"></div>
</body>
</html>
```

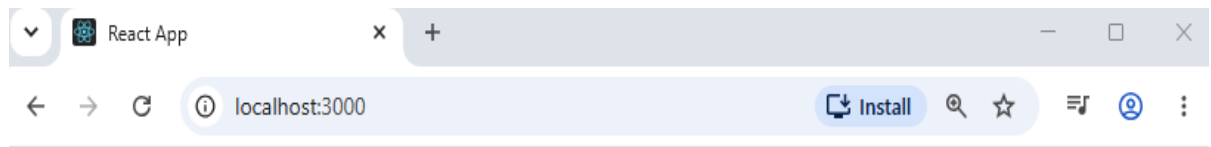
App.js

```
import React from 'react';
function App() {
  const name = "AtriDatta";
  const branch = "CSE";
  return (
    <div>
      <h1>Student Details</h1>
      <p>Name: {name}</p>
      <p>Branch: {branch}</p>
      <button>View Details</button>
    </div>
  );
}
export default App;
```

index.js

```
import ReactDOM from 'react-dom/client';
import App from './App';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

OUTPUT:



Student Details

Name: AtriDatta

Branch: CSE

[View Details](#)

3. Develop a program for creating and nesting components (function and class).

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>React App</title>
</head>
<body>
  <div id="root"></div>
</body>
</html>
```

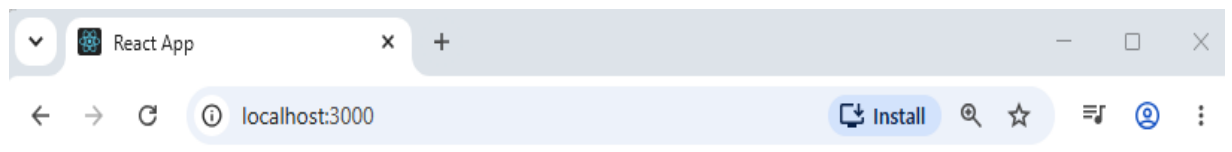
App.js

```
import React from 'react';
function Header() {
  return <h1>Student Portal</h1>;
}
class Student extends React.Component {
  render() {
    return <p>Student Name: Ravi</p>;
  }
}
function App() {
  return (
    <div>
      <Header />
      <Student />
    </div>
  );
}
export default App;
```

index.js

```
import ReactDOM from 'react-dom/client';  
import App from './App';  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<App />);
```

OUTPUT:



Student Portal

Student Name: Ravi

Experiment-6

1. Develop a program to work with props and states.

App.js

```
import React, { useState } from "react";
function Student(props) {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>Student Details</h2>
      <p>Name: {props.name}</p>
      <p>Branch: {props.branch}</p>
      <p>Button Clicks: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Click Me
      </button>
    </div>
  );
}
function App() {
  return (
    <div>
      <h1>Props and State Example</h1>
      <Student name="Ravi" branch="CSE" />
    </div>
  );
}

export default App;
```

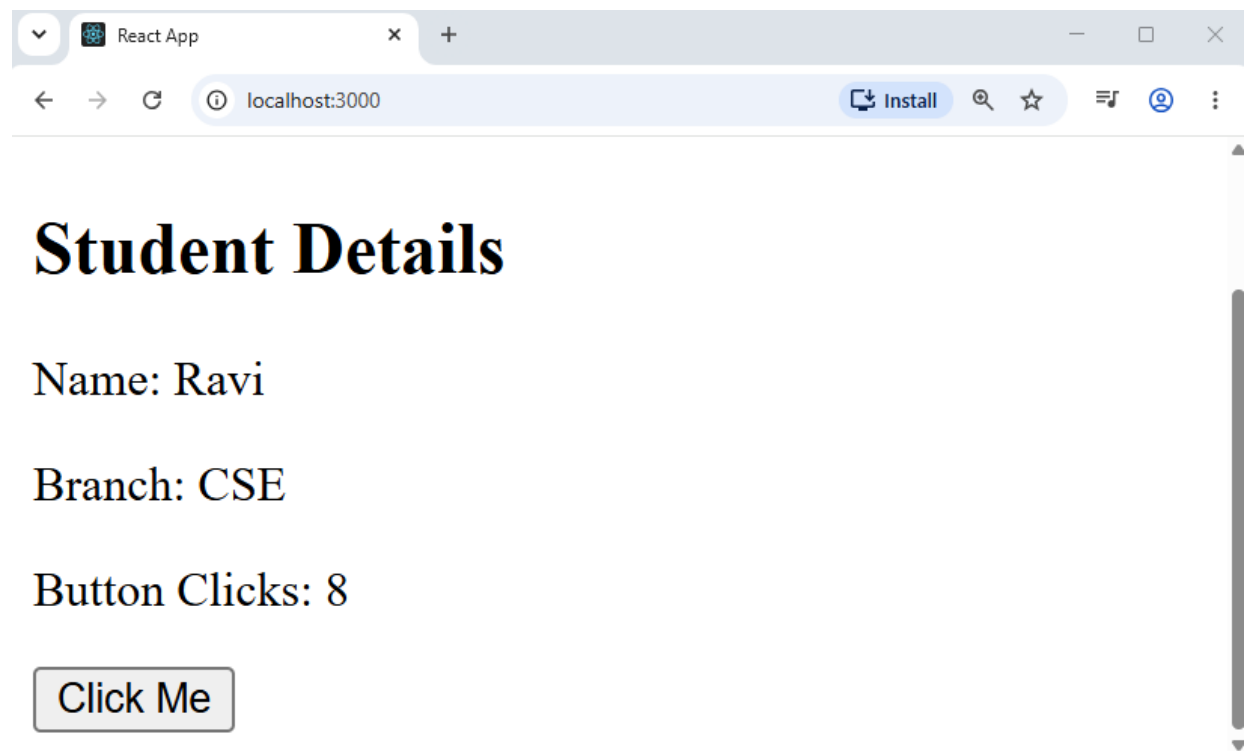
Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>React App</title>
</head>
<body>
  <div id="root"></div>
</body>
</html>
```

Index.js

```
import ReactDOM from 'react-dom/client';
import App from './App';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

OUTPUT:



2. Develop a program to add styles (CSS & Sass Styling) and display data.

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>React App</title>
</head>
<body>
  <div id="root"></div>
</body>
</html>
```

Index.js

```
import ReactDOM from 'react-dom/client';
import App from './App';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

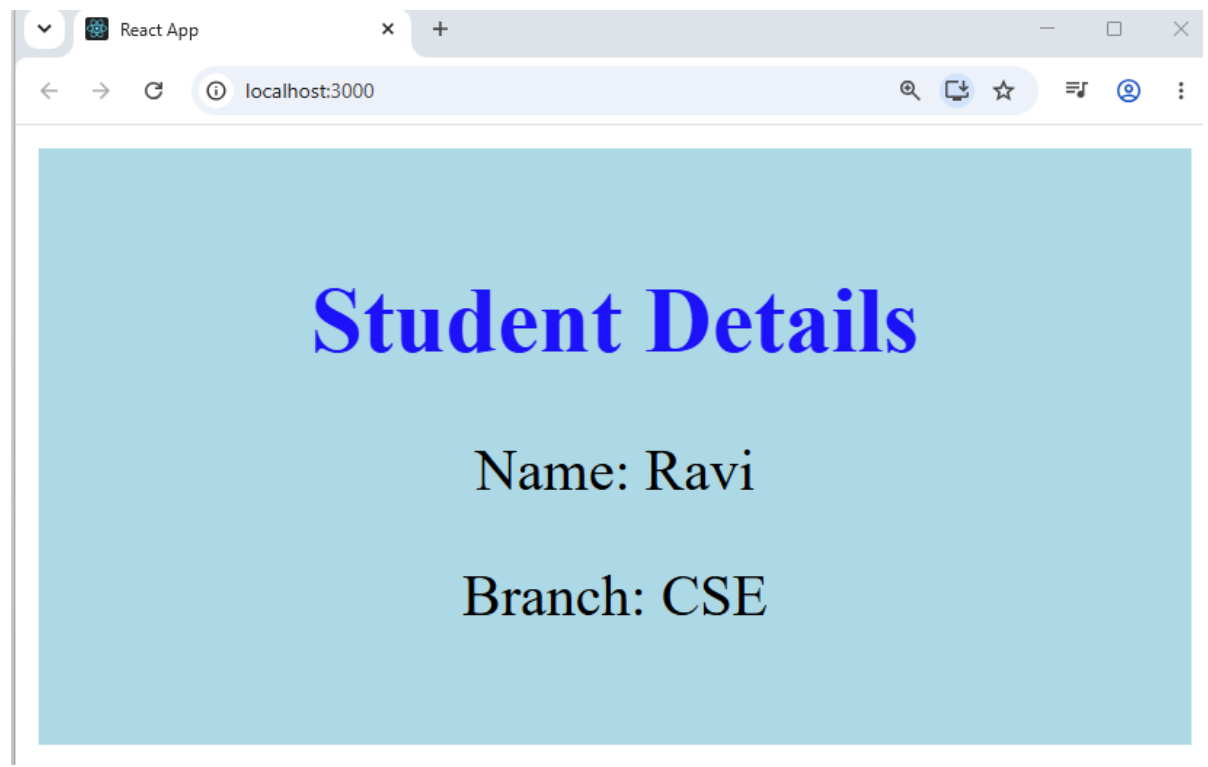
App.js

```
import React from "react";
import "./App.css";
function App() {
  const name = "Ravi";
  const branch = "CSE";
  return (
    <div className="box">
      <h1>Student Details</h1>
      <p>Name: {name}</p>
      <p>Branch: {branch}</p>
    </div>
  );
}
export default App;
```

App.css

```
.box {  
  text-align: center;  
  background-color: lightblue;  
  padding: 20px;  
}  
h1 {  
  color: blue;  
}  
p {  
  font-size: 20px;  
}
```

Output



Sass Styling

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>React App</title>
</head>
<body>
  <div id="root"></div>
</body>
</html>
```

Index.js

```
import ReactDOM from 'react-dom/client';
import App from './App';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

App.js

```
import React from "react";
import "./App.scss";
function App() {
  const name = "Ravi";
  const branch = "CSE";
  return (
    <div className="student">
      <h1>Student Details</h1>
      <p>Name: {name}</p>
      <p>Branch: {branch}</p>
    </div>
  );
}
export default App;
```

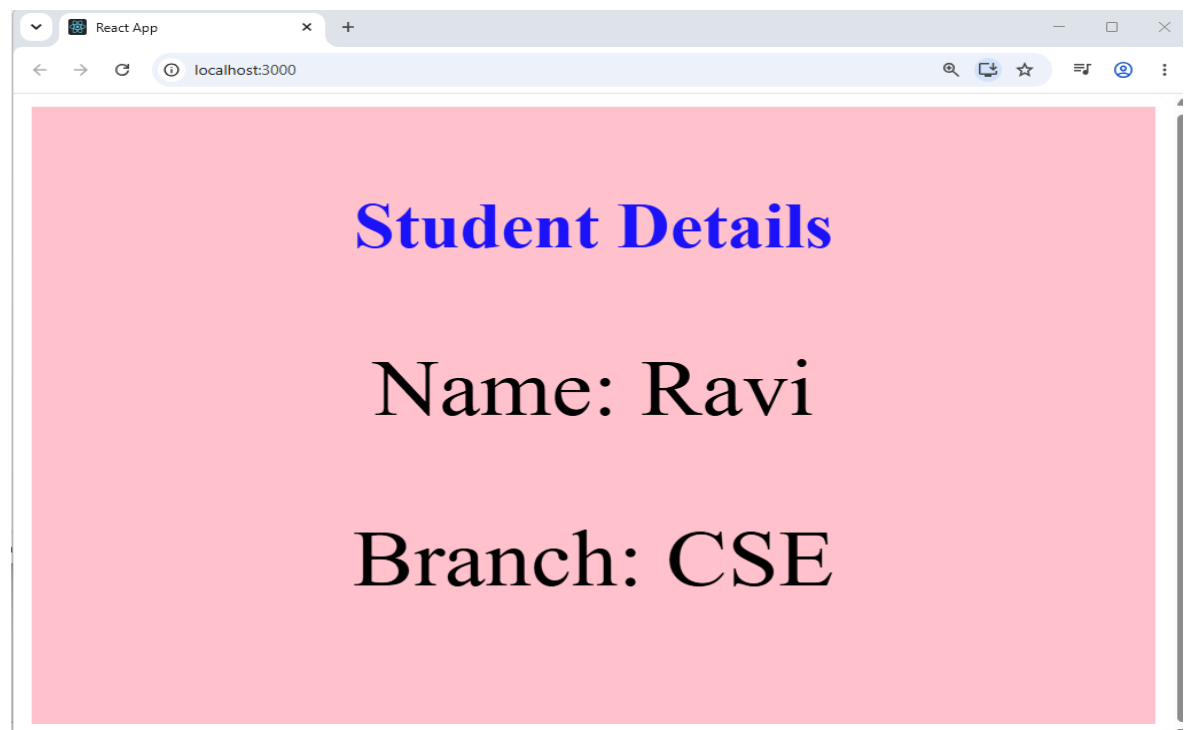
App.scss

```
$color: blue;

.student {
  text-align: center;
  background-color: lightblue;
  padding: 20px;

  h1 {
    color: $color;
  }
  p {
    font-size: 20px;
  }
}
```

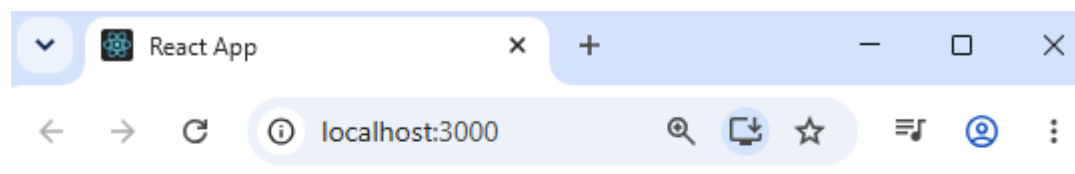
OUTPUT:



3. Development a program for responding to events.

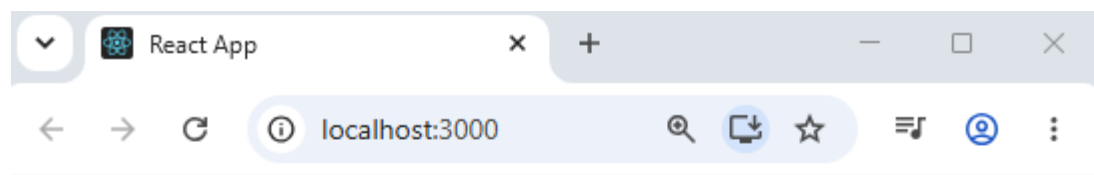
```
import React, { useState } from "react";  
function App() {  
    const [message, setMessage] = useState("Hello");  
    function handleClick() {  
        setMessage("Button Clicked!");  
    }  
    return (  
        <div> <h1>{message}</h1> <button onClick={handleClick}> Click Me </button>  
</div>  
    );  
}  
export default App;
```

OUTPUT:



Hello

Click Me



Button Clicked!

Click Me

Experiment-7

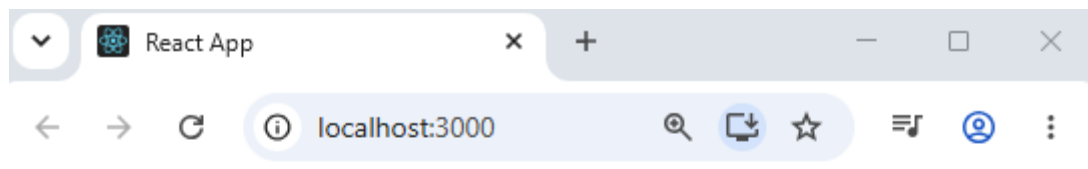
1. Develop a program for conditional rendering.

```
import React, { useState } from "react";

function App() {
  const [login, setLogin] = useState(false);
  return (
    <div>
      <h1>
        {login ? "Welcome User" : "Please Login"}
      </h1>
      <button onClick={() => setLogin(!login)}>
        {login ? "Logout" : "Login"}
      </button>
    </div>
  );
}

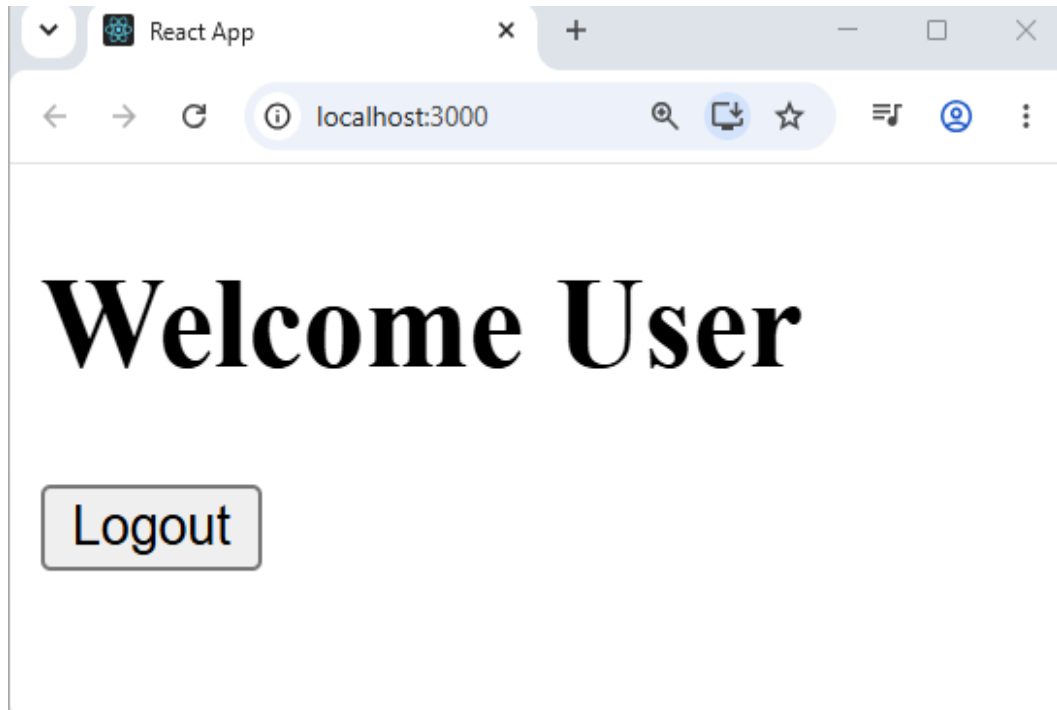
export default App;
```

OUTPUT:



Please Login

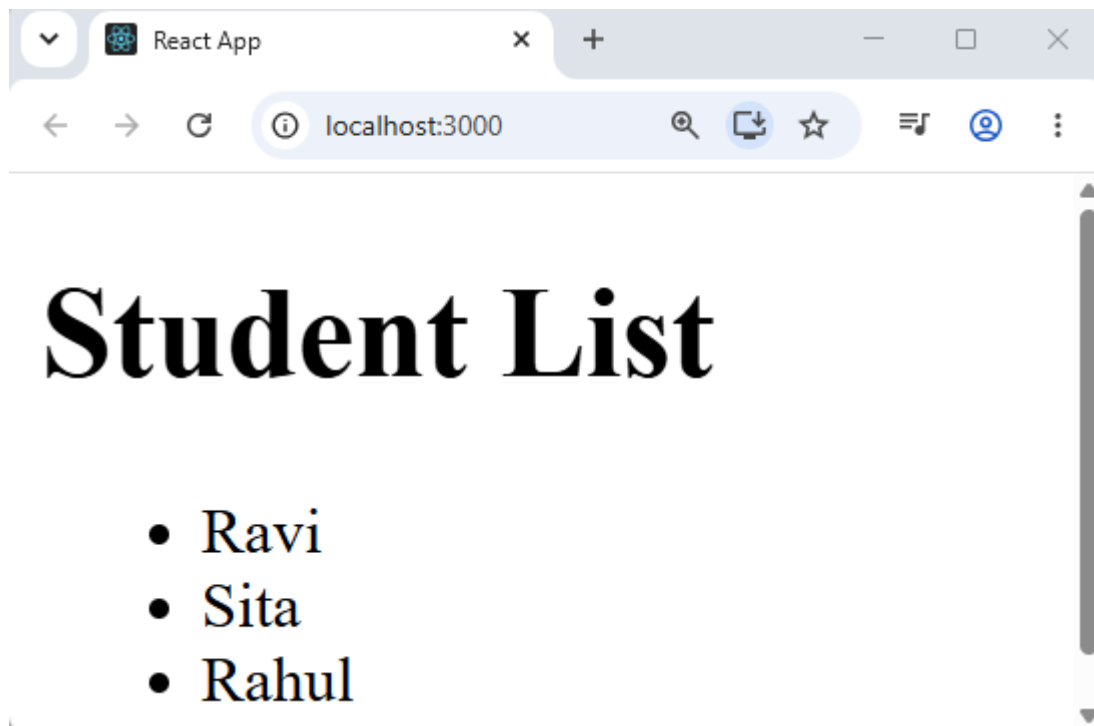
Login



2. Develop a program for rendering lists.

```
import React from "react";  
function App() {  
  const students = ["Ravi", "Sita", "Rahul"];  
  return (  
    <div>  
      <h1>Student List</h1>  
      <ul> {students.map((student) => ( <li key={student}>{student}</li> ))} </ul>  
    </div> );  
  }  
  export default App;
```

OUTPUT:



3. Develop a program for working with different form fields using React Forms.

App.js

```
import React, { useState } from "react";

function App() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [gender, setGender] = useState("");
  const [branch, setBranch] = useState("");

  function handleSubmit(e) {
    e.preventDefault();

    alert(
      "Name: " + name +
      "\nEmail: " + email +
      "\nGender: " + gender +
      "\nBranch: " + branch
    );
  }

  return (
    <div>
      <h1>Student Form</h1>

      <form onSubmit={handleSubmit}>

        <label>Name:</label>
        <input
          type="text"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />

        <br /><br />
      </form>
    </div>
  );
}
```

```
<label>Email:</label>
```

```
<input  
  type="email"  
  value={email}  
  onChange={(e) => setEmail(e.target.value)}  
>
```

```
<br /><br />
```

```
<label>Gender:</label>
```

```
<input  
  type="radio"  
  value="Male"  
  name="gender"  
  onChange={(e) => setGender(e.target.value)}  
>
```

Male

```
<input  
  type="radio"  
  value="Female"  
  name="gender"  
  onChange={(e) => setGender(e.target.value)}  
>
```

Female

```
<br /><br />
```

```
<label>Branch:</label>
```

```
<select  
  value={branch}  
  onChange={(e) => setBranch(e.target.value)}
```

```

    >
    <option value="">Select Branch</option>
    <option value="CSE">CSE</option>
    <option value="ECE">ECE</option>
    <option value="IT">IT</option>
  </select>
<br /><br />

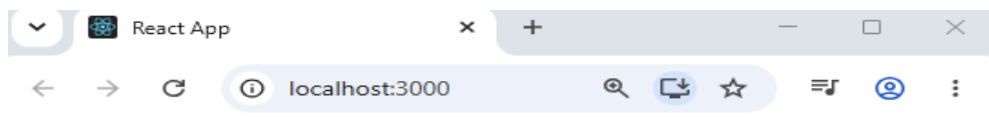
  <button type="submit">Submit</button>

</form>
</div>
);
}

```

export default App;

OUTPUT:



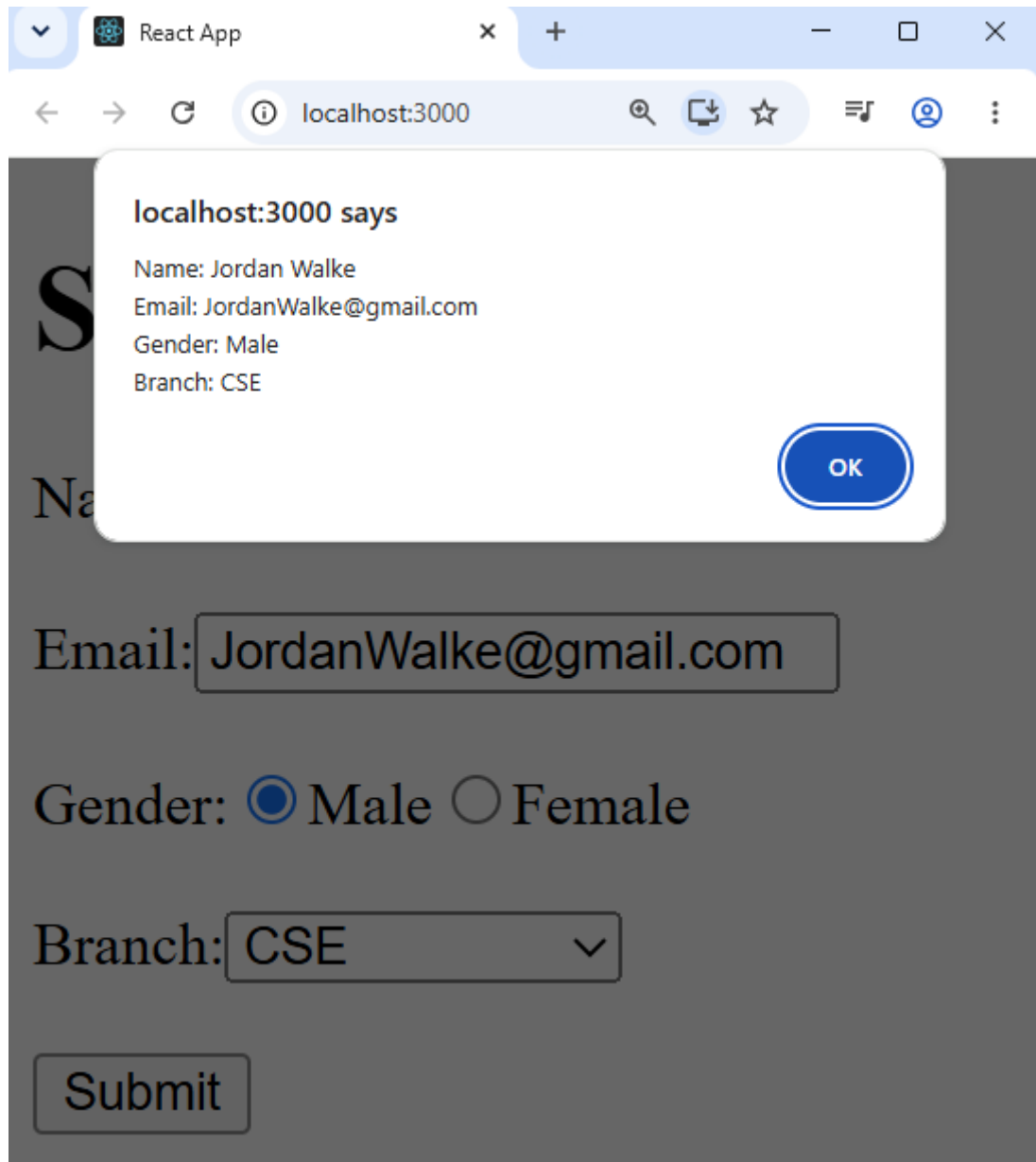
Student Form

Name:

Email:

Gender: ☒ Male ☐ Female

Branch: ▼



Experiment-8

1. Develop a program for routing to different pages using React Router.

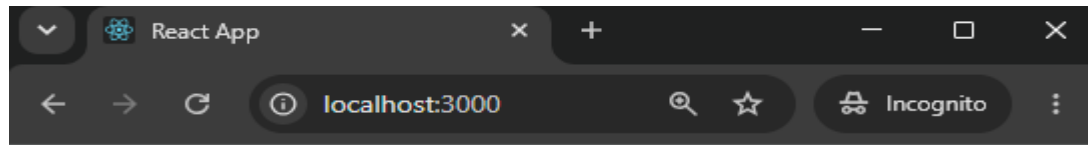
Step 1: Install React Router

```
npm install react-router-dom
```

Step 2: App.js

```
import React from "react";
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
function Home() {
    return <h2>Home Page</h2>;
}
function About() {
    return <h2>About Page</h2>;
}
function Contact() {
    return <h2>Contact Page</h2>;
}
function App() {
    return (
        <BrowserRouter>
            <h1>My Website</h1>
            <Link to="/">Home</Link> |{" "}
            <Link to="/about">About</Link> |{" "}
            <Link to="/contact">Contact</Link>
            <Routes>
                <Route path="/" element={<Home />} />
                <Route path="/about" element={<About />} />
                <Route path="/contact" element={<Contact />} />
            </Routes>
        </BrowserRouter>
    );
}
export default App;
```

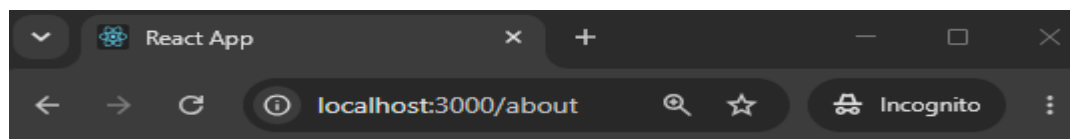
OUTPUT:



My Website

[Home](#) | [About](#) | [Contact](#)

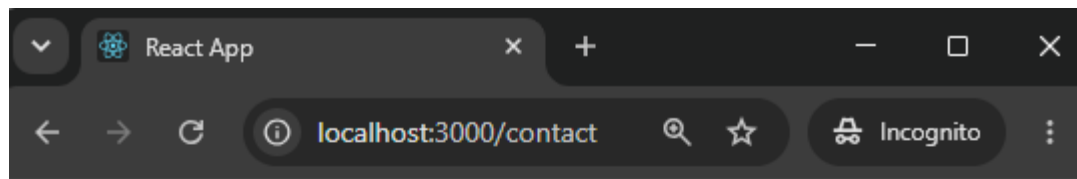
Home Page



My Website

[Home](#) | [About](#) | [Contact](#)

About Page



My Website

[Home](#) | [About](#) | [Contact](#)

Contact Page

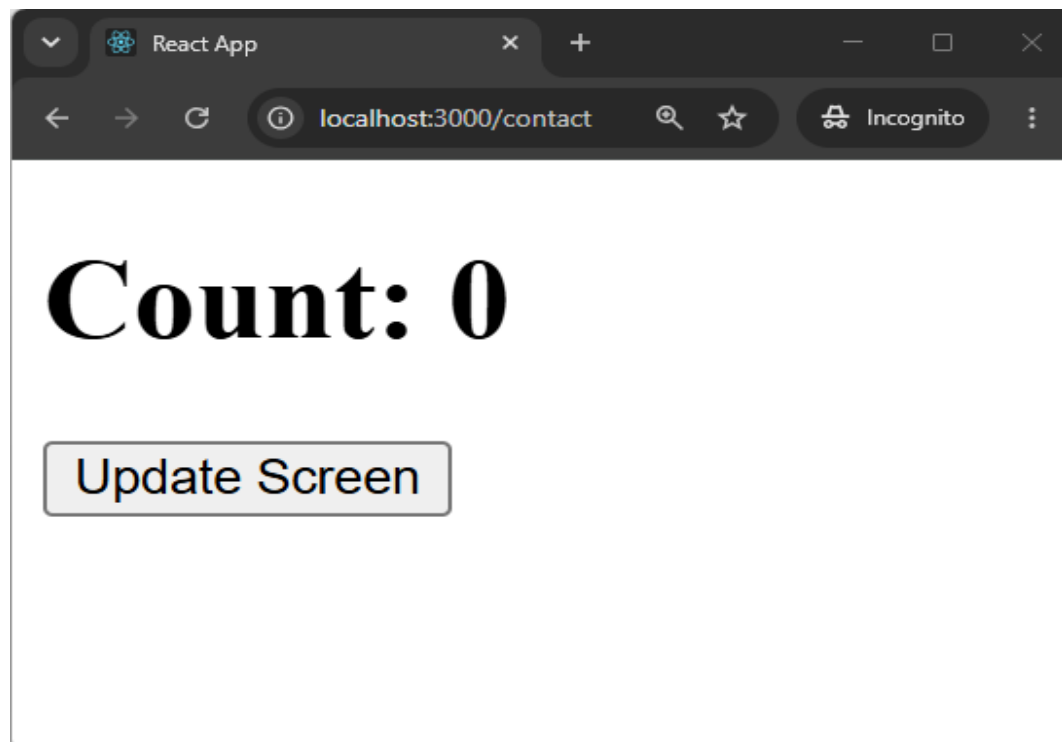
2. Develop a program for updating the screen.

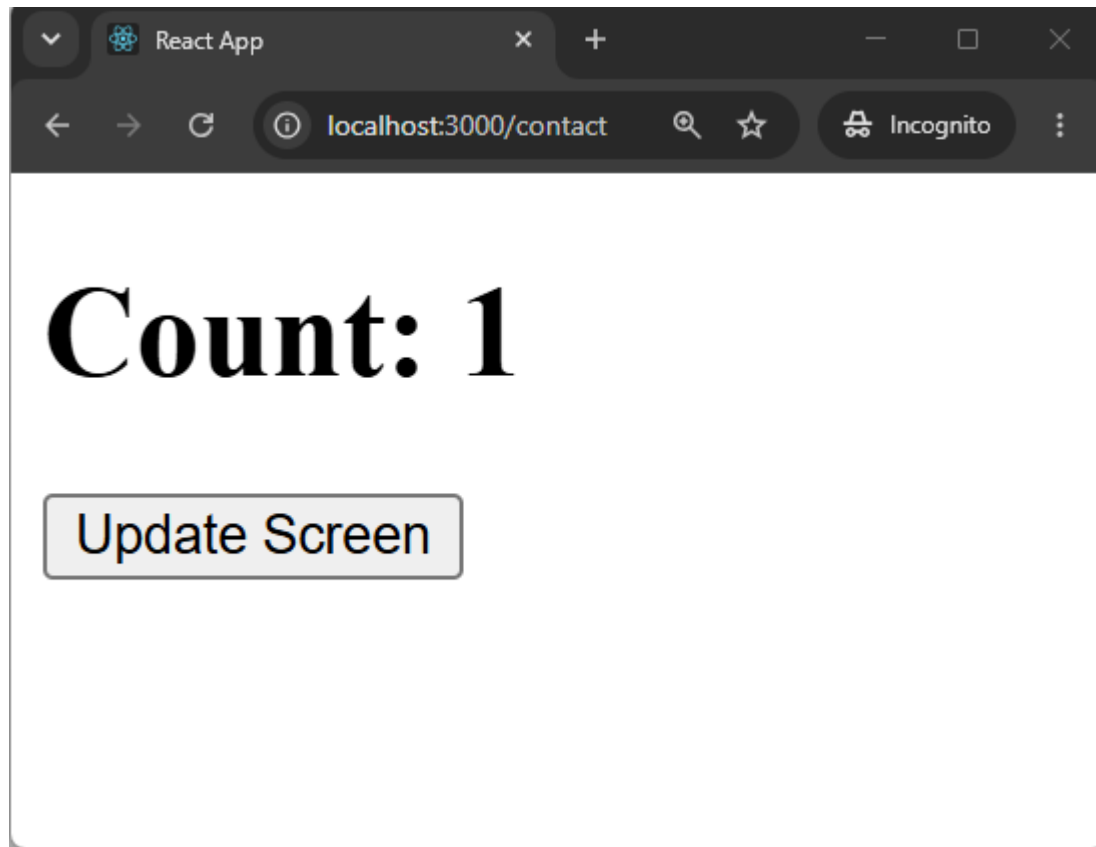
App.js

```
import React, { useState } from "react";
function App() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h1>Count: {count}</h1>

      <button onClick={() => setCount(count + 1)}>
        Update Screen
      </button>
    </div>
  );
}
export default App;
```

OUTPUT:





Experiment-9

1. Develop a program to understand the importance of using Hooks.

App.js

```
import React, {
  useState,
  useEffect,
  useContext,
  useRef,
  useMemo,
  useCallback,
  useReducer,
  createContext
} from "react";
const StudentContext = createContext();
function counterReducer(state, action) {
  switch (action.type) {
    case "increase":
      return { count: state.count + 1 };

    case "decrease":
      return { count: state.count - 1 };

    default:
      return state;
  }
}
function App() {
  const [name, setName] = useState("Ravi");
  const [marks, setMarks] = useState(80);
  const inputRef = useRef(null);
  const [state, dispatch] = useReducer(counterReducer, {
    count: 0
  });
}
```

```

useEffect(() => {
  document.title = "Student: " + name;
}, [name]);
const student = {
  name: name,
  branch: "CSE",
  marks: marks
};
const average = useMemo(() => {
  return marks / 1;
}, [marks]);
const focusInput = useCallback(() => {
  inputRef.current.focus();
}, []);
return (
  <StudentContext.Provider value={student}>
    <div>
      <h1>Student Dashboard</h1>
      <h2>Student Name: {name}</h2>
      <h2>Branch: {student.branch}</h2>
      <h2>Marks: {marks}</h2>
      <h2>Average: {average}</h2>
      <input
        ref={inputRef}
        type="text"
        placeholder="Enter student name"
        onChange={(e) => setName(e.target.value)}
      />
      <br /><br />
      <button onClick={focusInput}>
        Focus Input
      </button>
      <br /><br />
    </div>
  </StudentContext.Provider>
)

```

```

    <button onClick={() => setMarks(marks + 5)}>
      Increase Marks
    </button>

  <Student />

  <h2>Counter: {state.count}</h2>

  <button onClick={() => dispatch({ type: "increase" })}>
    +
  </button>

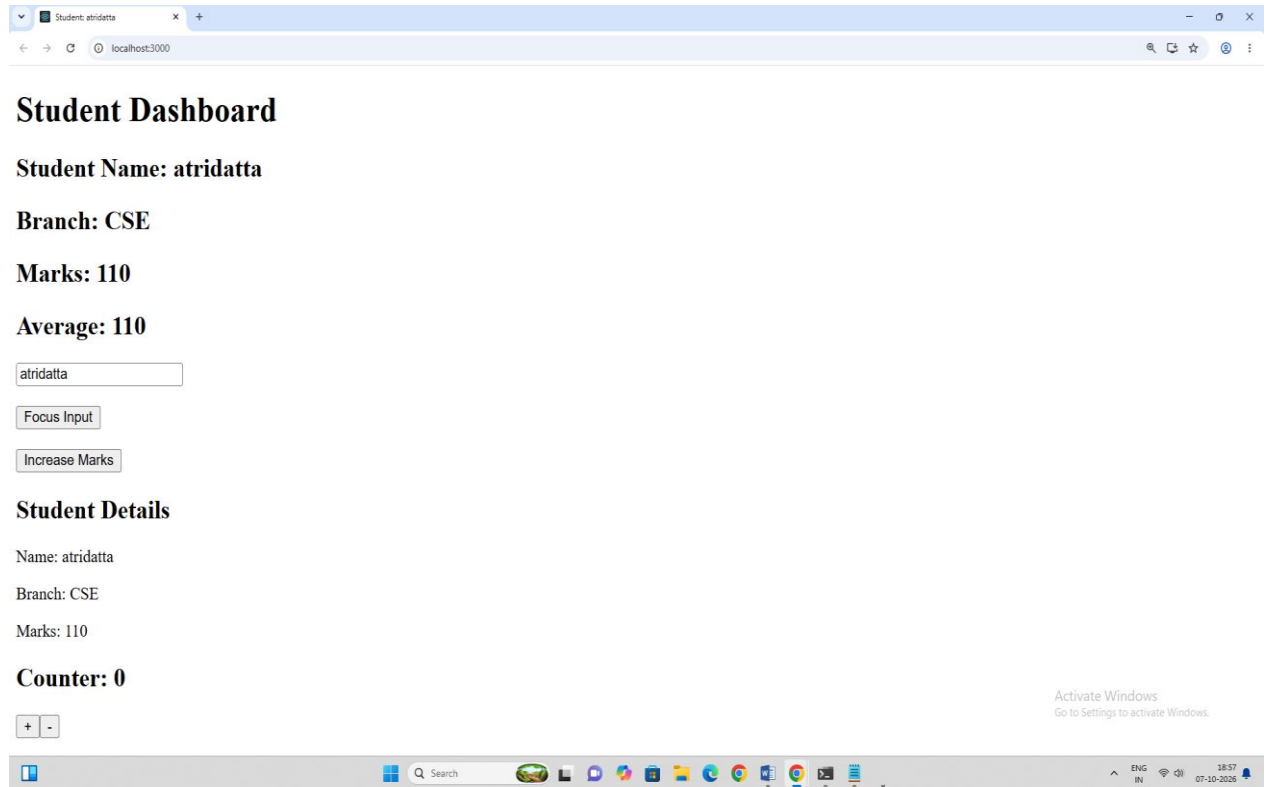
  <button onClick={() => dispatch({ type: "decrease" })}>
    -
  </button>
</div>
</StudentContext.Provider>
);
}

function Student() {
  const student = useContext(StudentContext);
  return (
    <div>
      <h2>Student Details</h2>
      <p>Name: {student.name}</p>
      <p>Branch: {student.branch}</p>
      <p>Marks: {student.marks}</p>
    </div>
  );
}

export default App;

```

OUTPUT:



2. Develop a program for sharing data between components.

App.js

```
import React, { createContext, useContext } from "react";
```

```
const StudentContext = createContext();
```

```
function App() {
```

```
  const student = {  
    name: "Ravi",  
    branch: "CSE",  
    year: "III"  
  };
```

```
  return (  
    <StudentContext.Provider value={student}>  
      <h1>Student Information</h1>  
      <Student />  
    </StudentContext.Provider>  
  );  
}
```

```
function Student() {
```

```
  return (  
    <div>  
      <StudentDetails />  
    </div>  
  );  
}
```

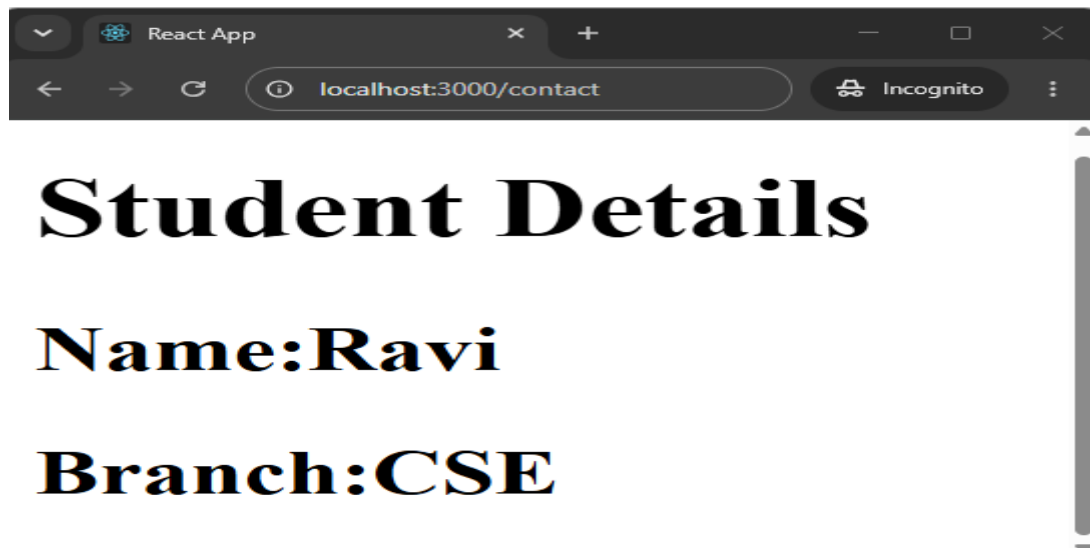
```
function StudentDetails() {
```

```
const student = useContext(StudentContext);

return (
  <div>
    <h2>Name: {student.name}</h2>
    <h2>Branch: {student.branch}</h2>
    <h2>Year: {student.year}</h2>
  </div>
);
}
```

export default App;

OUTPUT:



Experiment-10

a. Install MongoDB and Configure ATLAS

Steps for Installation & Configuration

1. Install MongoDB Community Server (Local):

- Download MongoDB Community Server installer for your OS from the official website.
- Run the installer and select **Complete** installation.
- Install **MongoDB Compass** (GUI) and ensure **Install MongoDB as a Service** is checked.
- Verify installation in terminal/command prompt:

```
mongod --version
```

2. Configure MongoDB Atlas (Cloud Database):

- **Step 1:** Sign up or log in to mongodb.com.
- **Step 2:** Click **Build a Database** and select the **M0 Free Tier**.
- **Step 3:** Create a **Database User** with a username and password under **Database Access**.
- **Step 4:** Navigate to **Network Access**, click **Add IP Address**, and select **Allow Access from Anywhere** (0.0.0.0/0).
- **Step 5:** Click **Connect** on your cluster, select **Connect with the MongoDB Shell (mongosh)**, and copy your connection string:

```
mongodb+srv://<username>:<password>@cluster0.mongodb.net/studentdb
```

OUTPUT:

Plaintext

Connecting to MongoDB Atlas Cluster...

Current Mongosh Log Directory: C:\Users\Admin\.mongodb\mongosh

Connecting to: mongodb+srv://admin:xxxx@cluster0.mongodb.net/studentdb

Using MongoDB: 7.0.5

Atlas Cluster0-shard-00-00 [primary] studentdb>

Connection Successful! Connected to MongoDB Atlas.

b. Develop MongoDB queries to perform CRUD operations on document using insert(), find(), update(), remove()

MongoDB Queries (mongosh)

1. Switch or Create Database

use studentdb;

2. Insert Operations (CREATE)

db.students.insert({ id: 101, name: "Ravi", branch: "CSE", year: 3 });

```
db.students.insert([
  { id: 102, name: "Sita", branch: "ECE", year: 2 },
  { id: 103, name: "Rahul", branch: "IT", year: 4 }
]);
```

3. Find Operations (READ)

Retrieve all documents

db.students.find();

Retrieve specific document using query filter

db.students.find({ branch: "CSE" });

4. Update Operations (UPDATE)

Update branch of student with id 101

```
db.students.update(
  { id: 101 },
  { $set: { branch: "CST" } }
);
```

Verify updated document

db.students.find({ id: 101 });

5. Remove Operations (DELETE)

Remove student with id 103

db.students.remove({ id: 103 });

Verify remaining documents

db.students.find();

OUTPUT:

switched to db studentd

```

> db.students.insert({ id: 101, name: "Ravi", branch: "CSE", year: 3 });
WriteResult({ "nInserted" : 1 })
> db.students.insert([ { id: 102, name: "Sita", branch: "ECE", year: 2 }, { id: 103, name: "Rahul",
branch: "IT", year: 4 } ]);
BulkWriteResult({
  "writeErrors" : [ ],
  "writeConcernErrors" : [ ],
  "nInserted" : 2,
  "nUpserted" : 0,
  "nMatched" : 0,
  "nModified" : 0,
  "nRemoved" : 0,
  "upserted" : [ ]
})
> db.students.find();
[
  { _id: ObjectId("65e01234a1b2c3d4e5f60001"), id: 101, name: 'Ravi', branch: 'CSE', year: 3 },
  { _id: ObjectId("65e01234a1b2c3d4e5f60002"), id: 102, name: 'Sita', branch: 'ECE', year: 2 },
  { _id: ObjectId("65e01234a1b2c3d4e5f60003"), id: 103, name: 'Rahul', branch: 'IT', year: 4 }
]

> db.students.find({ branch: "CSE" });
[
  { _id: ObjectId("65e01234a1b2c3d4e5f60001"), id: 101, name: 'Ravi', branch: 'CSE', year: 3 }
]

> db.students.update({ id: 101 }, { $set: { branch: "CST" } });
WriteResult({ "nMatched" : 1, "nInserted" : 0, "nModified" : 1 })

> db.students.find({ id: 101 });
[
  { _id: ObjectId("65e01234a1b2c3d4e5f60001"), id: 101, name: 'Ravi', branch: 'CST', year: 3 }
]

> db.students.remove({ id: 103 });

```

```
WriteResult({ "nRemoved" : 1 })
```

```
> db.students.find();
```

```
[  
  { _id: ObjectId("65e01234a1b2c3d4e5f60001"), id: 101, name: 'Ravi', branch: 'CST', year: 3 },  
  { _id: ObjectId("65e01234a1b2c3d4e5f60002"), id: 102, name: 'Sita', branch: 'ECE', year: 2 }  
]
```

Experiment-11

a. Develop MongoDB queries to Create and drop databases and collections.

1. Create a database

(MongoDB creates a database implicitly when you switch to it and insert data)

use collegedb;

2. Create a collection explicitly

db.createCollection("faculty");

3. Show all collections in the current database

show collections;

4. Drop a collection

db.faculty.drop();

5. Drop the current database

db.dropDatabase();

OUTPUT:

> use collegedb

switched to db collegedb

> db.createCollection("faculty")

{ "ok" : 1 }

> show collections

faculty

> db.faculty.drop()

true

> db.dropDatabase()

{ "dropped" : "collegedb", "ok" : 1 }

b. Develop MongoDB queries to work with records using find(), limit(), sort(), createIndex(), aggregate().

Switch to the database and insert sample records for demonstration

use studentdb;

```
db.students.insertMany([
  { id: 1, name: "Ravi", branch: "CSE", score: 85 },
  { id: 2, name: "Sita", branch: "ECE", score: 92 },
  { id: 3, name: "Rahul", branch: "IT", score: 78 },
  { id: 4, name: "Priya", branch: "CSE", score: 88 },
  { id: 5, name: "Amit", branch: "ECE", score: 75 }
]);
```

1. find() - Retrieve all records

```
db.students.find();
```

2. limit() - Limit the result to the first 2 records

```
db.students.find().limit(2);
```

3. sort() - Sort records by score in descending order (-1)

```
db.students.find().sort({ score: -1 });
```

4. createIndex() - Create an ascending index (1) on the 'name' field

```
db.students.createIndex({ name: 1 });
```

5. aggregate() - Group by branch and calculate the average score and student count

```
db.students.aggregate([
  {
    $group: {
      _id: "$branch",
      avgScore: { $avg: "$score" },
      totalStudents: { $sum: 1 }
    }
  }
]);
```

OUTPUT:

```
> use studentdb
```

```
switched to db studentdb
```

```
> db.students.insertMany([ { id: 1, name: "Ravi", branch: "CSE", score: 85 }, ... ])
{
  "acknowledged" : true,
  "insertedIds" : [ ObjectId("65e012...01"), ObjectId("65e012...02"), ObjectId("65e012...03"),
ObjectId("65e012...04"), ObjectId("65e012...05") ]
}
```

```
> db.students.find();
[
  { _id: ObjectId("..."), id: 1, name: 'Ravi', branch: 'CSE', score: 85 },
  { _id: ObjectId("..."), id: 2, name: 'Sita', branch: 'ECE', score: 92 },
  { _id: ObjectId("..."), id: 3, name: 'Rahul', branch: 'IT', score: 78 },
  { _id: ObjectId("..."), id: 4, name: 'Priya', branch: 'CSE', score: 88 },
  { _id: ObjectId("..."), id: 5, name: 'Amit', branch: 'ECE', score: 75 }
]
```

```
> db.students.find().limit(2);
[
  { _id: ObjectId("..."), id: 1, name: 'Ravi', branch: 'CSE', score: 85 },
  { _id: ObjectId("..."), id: 2, name: 'Sita', branch: 'ECE', score: 92 }
]
```

```
> db.students.find().sort({ score: -1 });
[
  { _id: ObjectId("..."), id: 2, name: 'Sita', branch: 'ECE', score: 92 },
  { _id: ObjectId("..."), id: 4, name: 'Priya', branch: 'CSE', score: 88 },
  { _id: ObjectId("..."), id: 1, name: 'Ravi', branch: 'CSE', score: 85 },
  { _id: ObjectId("..."), id: 3, name: 'Rahul', branch: 'IT', score: 78 },
  { _id: ObjectId("..."), id: 5, name: 'Amit', branch: 'ECE', score: 75 }
]
```

```
> db.students.createIndex({ name: 1 });
```

name_1

```
> db.students.aggregate([ { $group: { _id: "$branch", avgScore: { $avg: "$score" }, totalStudents: { $sum: 1 } } } ] );  
[  
  { _id: 'IT', avgScore: 78, totalStudents: 1 },  
  { _id: 'ECE', avgScore: 83.5, totalStudents: 2 },  
  { _id: 'CSE', avgScore: 86.5, totalStudents: 2 }  
]
```